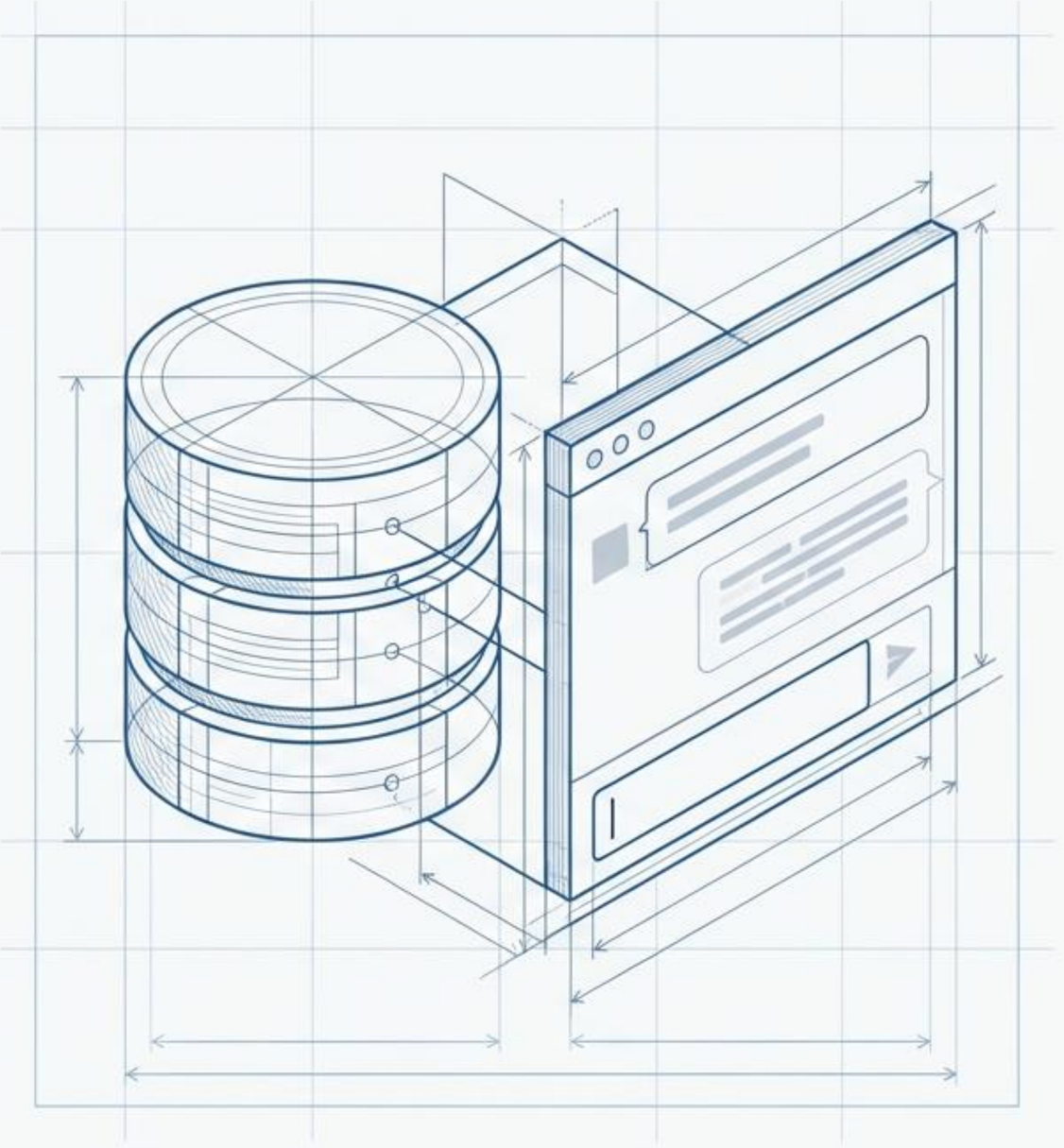
Building an Autonomous AI SQL Assistant in Oracle APEX

Converting natural language into executable Oracle SQL without writing a single line of code.

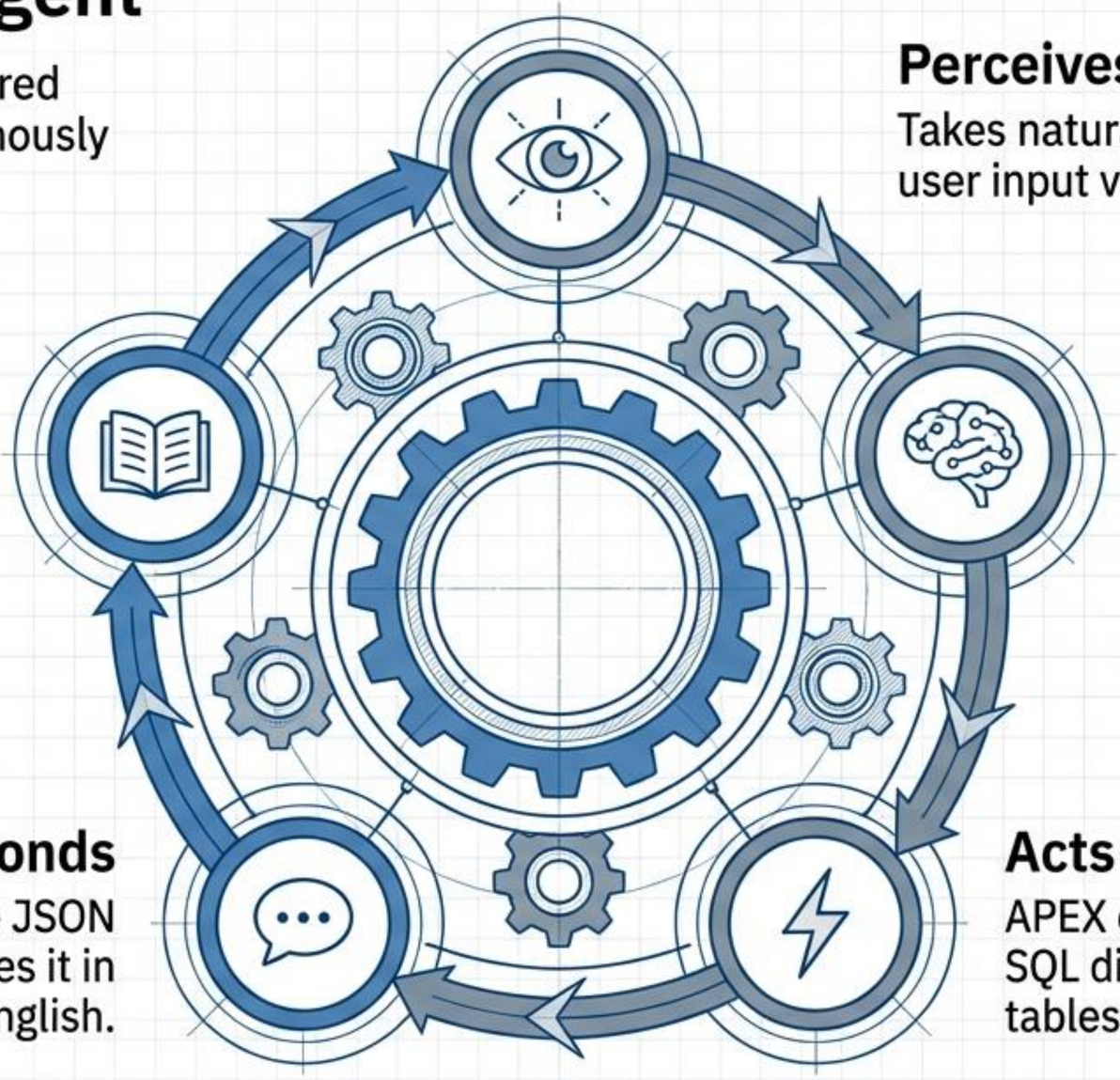
Author Credentials: Ravindra Thapliyal | Oracle ACE Associate

Context: Complete Technical Documentation & Architecture Blueprint



Beyond a Text Generator: Defining the AI Agent

This application is an AI-Powered SQL Assistant. It acts autonomously through five distinct phases:



Perceives

Takes natural language user input via APEX.

Reasons

The AI translates the intent into precise `Oracle SQL`.

Acts

APEX executes the generated SQL directly on live database tables.

Responds

The AI consumes the JSON output and summarizes it in plain English.

Learns

Context is preserved by logging every query in a dedicated audit table.

The Architecture Blueprint and Technology Stack

Frontend Layer

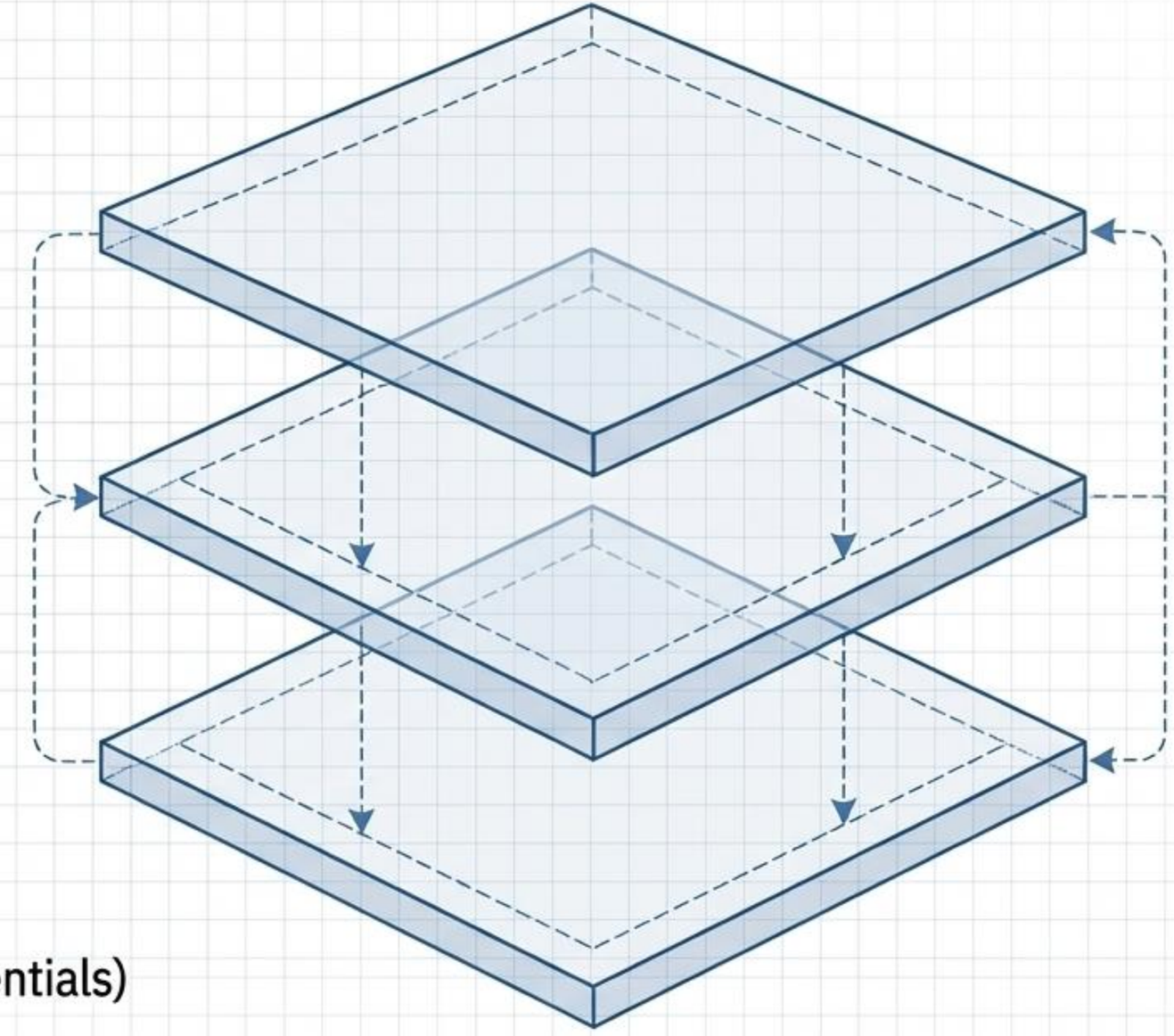
- Oracle APEX (Universal Theme)
- Dynamic Actions & JavaScript
- `Chart.js 4.4` (CDN loaded for dynamic bar charts)

Backend PL/SQL Engine

- `DBMS_SQL` for dynamic execution
- `APEX_WEB_SERVICE` for API calls

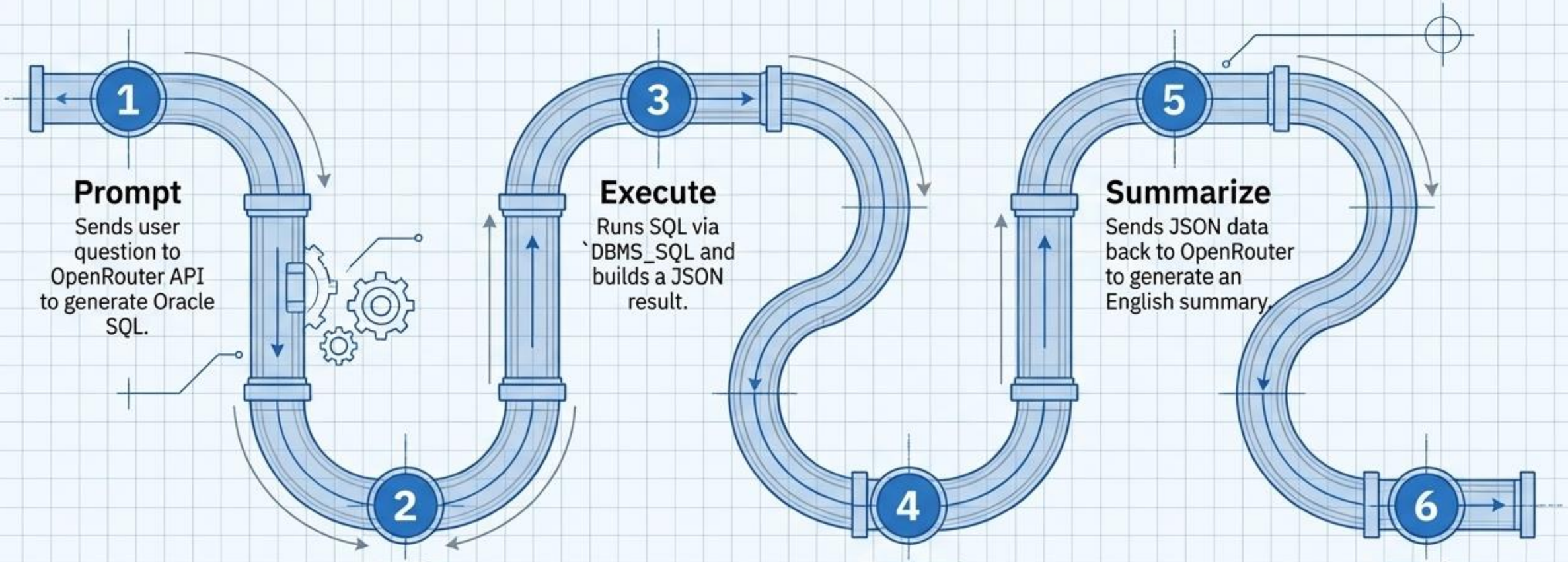
AI & Database Layer

- API: OpenRouter (inception/mercury-2 reasoning model)
- Database: Oracle DB (`SALES_ORDERS` and `AI_QUERY_LOG`)
- Security: APEX Credential Store (Web Credentials)



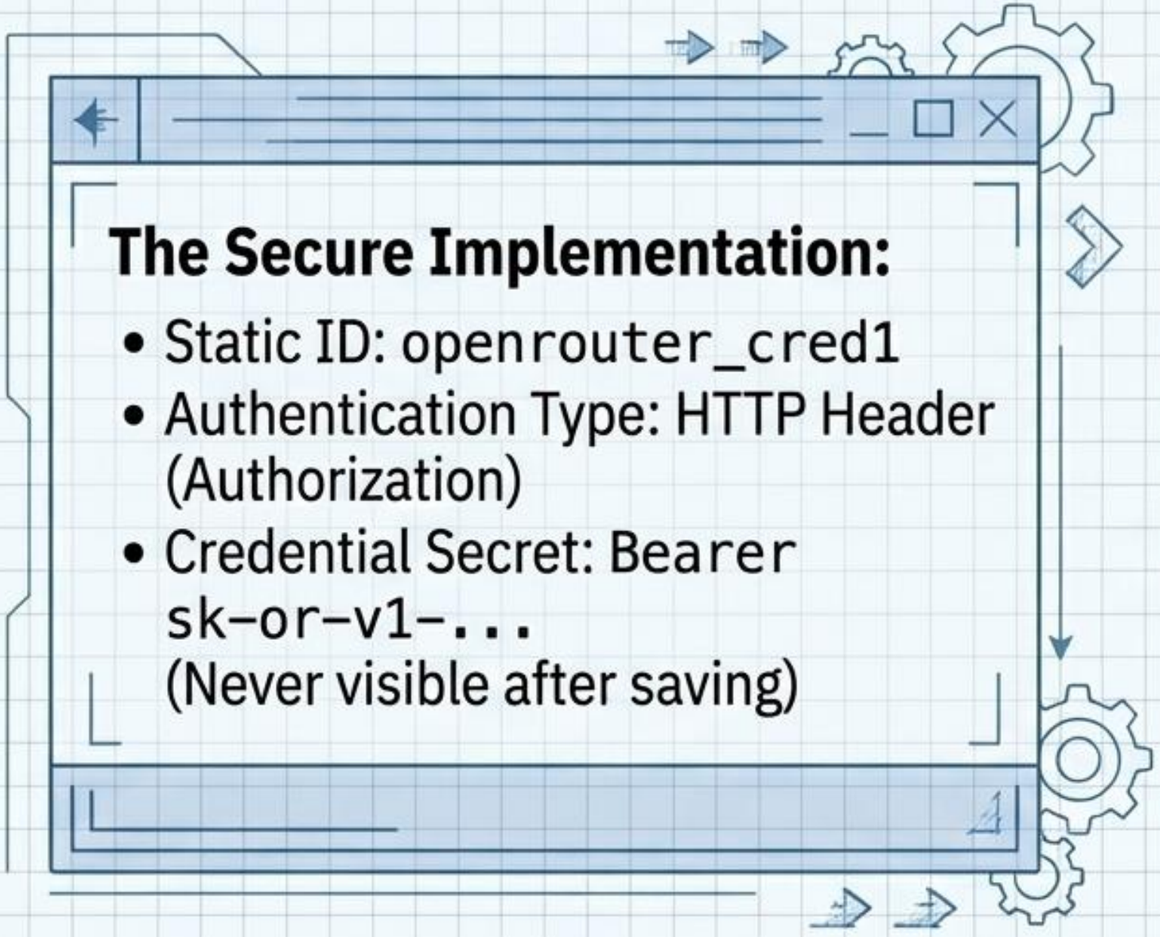
The 6-Step Asynchronous Processing Pipeline

Triggered entirely via **apex.server.process**. Zero page refreshes.



Enterprise-Grade Security with Web Credentials

The OpenRouter API key is never hardcoded in PL/SQL. It is stored securely in Shared Components → **Security** → **Web Credentials**.



The Secure Implementation:

- Static ID: `openrouter_cred1`
- Authentication Type: HTTP Header (Authorization)
- Credential Secret: `Bearer sk-or-v1-...` (Never visible after saving)

```
l_response := apex_web_service.make_rest_request(
    p_url                  => c_url,
    p_http_method          => 'POST',
    p_credential_static_id => 'openrouter_cred1',
    ...
);
```

Secure Reference

The Data Foundation and Audit Trail

Business Data: SALES_ORDERS

Contains 50 sample rows populated via Oracle's INSERT ALL pattern. Tracks Regions, Reps, Categories, Quantities, and Statuses.

ORDER_ID (NUMBER)	CUSTOMER_NAME (VARCHAR2)
CUSTOMER_NAME (VARCHAR2)	REP (VARCHAR2)
TOTAL_AMOUNT (NUMBER)	REP (VARCHAR2)
QUANTITY (NUMBER)	CUA[illegible]
STATUS (VARCHAR2)	

101	Acme Corp.	550.00	NA	Jones	Software	10	Shipped
102	Beta Industries	325.50	EU	Smith	Hardware	5	Pending
103	Gamma Solutions	800.25	ASIA	Lee	Services	2	Delivered

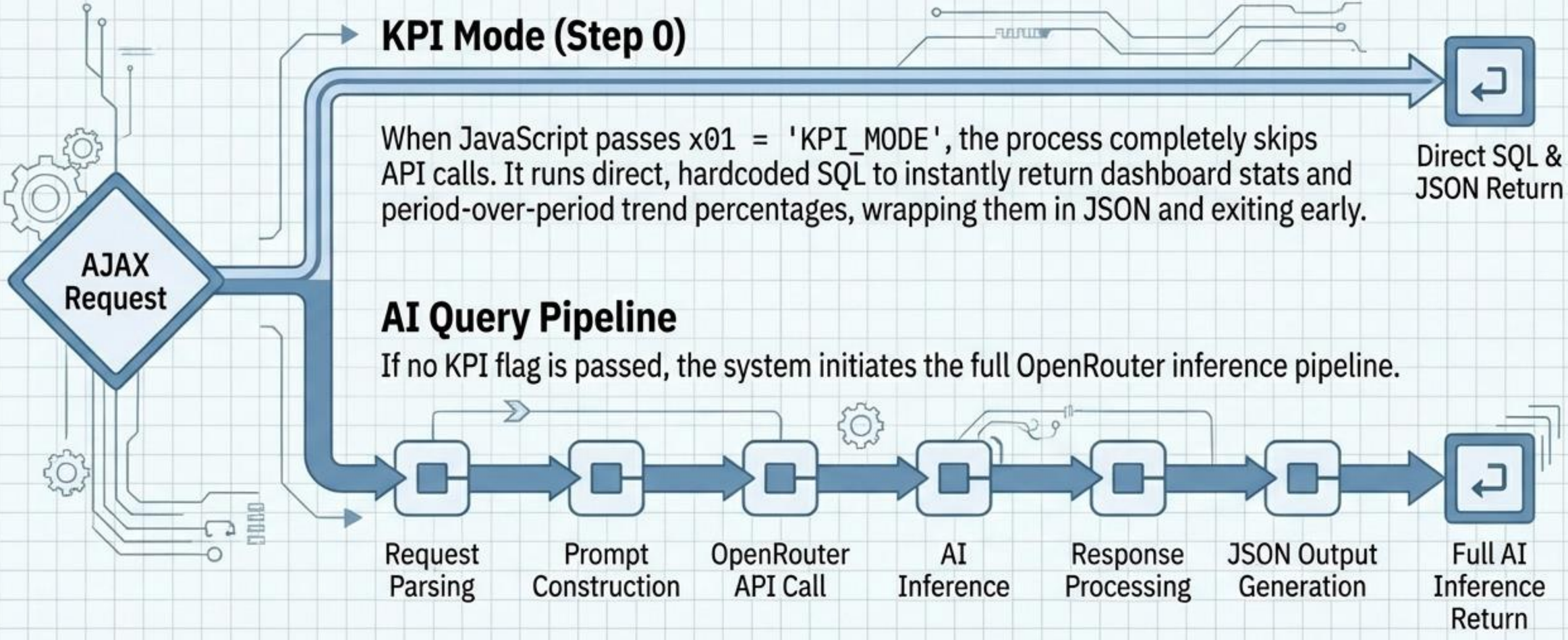
Audit Tracking: AI_QUERY_LOG

Every system interaction is captured to monitor AI behavior and token usage.

COLUMN_NAME	(DATA_TYPE)
• LOG_ID	NUMBER)
• TIMESTAMP	TIMESTAMP)
• USER_ID	VARCHAR2)
• PROMPT_TEXT	CLOB
• AI_GENERATED_SQL	CLOB
• EXECUTION_RESULT	CLOB
• SUMMARY_RESPONSE	CLOB
• TOKEN_USAGE	NUMBER)

Dual-Mode Processing Inside the AJAX Callback

The CALL_SQL_COPILOT_AI process handles two distinct workloads.



Step 1 & 2: Prompting the Reasoning Model and Safe Logging

The Model Context

The Inference Model

- Uses inception/mercury-2 via OpenRouter. This is a reasoning model that thinks internally before outputting.
- We enforce a strict system prompt demanding only raw SQL—no markdown, no explanations.

Safe Audit Logging

Every generated query is logged.

```
BEGIN
    INSERT INTO AI_QUERY_LOG...
EXCEPTION
    WHEN OTHERS THEN l_log_id := NULL;
END;
```

Architect's Rule: The INSERT into AI_QUERY_LOG must be wrapped in a BEGIN/EXCEPTION block. If the table fails (e.g., tablespace full), l_log_id becomes NULL, but the user's workflow continues uninterrupted.

Step 5 & 6: Data Summarization and JSON Construction

The Summarizer Persona

The executed data payload is sent back to `inception/mercury-2`. The system prompt now shifts the AI into a "Sales Data Analyst" role, instructed to return a concise, 10-line bulleted summary of the JSON data.

Fallback Mechanism

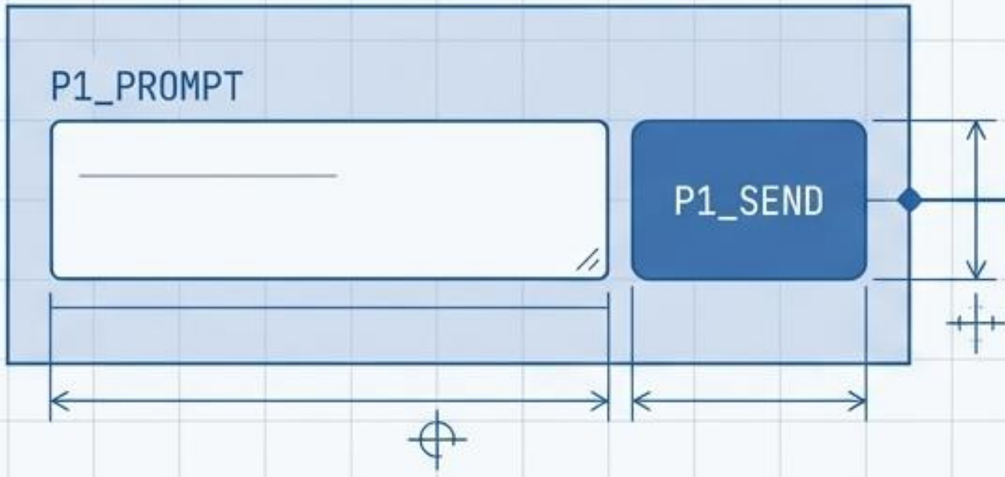
If the AI call fails, a PL/SQL fallback string is generated (e.g., "Query executed. **14 orders retrieved.**").

Final Payload

The final response packages `ai_reply`, `chart_title`, and **extracted** `chart_data` (using REGEXP_SUBSTR) into a single **JSON object** sent to the browser.

Frontend Orchestration and JavaScript Execution

The APEX UI is entirely driven by Static Content regions and Dynamic Actions tied to the P1_SEND button.



JAVASCRIPT EXECUTION

```
apex.server.process('CALL_SQL_COPILOT_AI', {
    dataType: 'text', // CRITICAL RULE
    success: function(rawResult) { ... }
});
```

Critical JavaScript Rule:

When firing `apex.server.process`, you must set `dataType: 'text'`. If omitted, JavaScript attempts a premature JSON parse before the server response is fully resolved, causing silent failures.

The Assistant in Action: Natural Language Querying

Simple (Basic Filters)

"Show all delivered orders" or
"Show all orders from North region"

Medium (Conditions & Joins)

"Show all Electronics orders where payment is paid" or
"Show cancelled orders with refunded payment"

Complex (Aggregations & Sorting)

"Show sales rep performance with total orders and total revenue ordered by revenue highest first"

Troubleshooting Guide and Final Takeaways

Error	Cause	Fix
ORA-01536	Tablespace full	Delete old AI_QUERY_LOG rows.
ORA-40597	JSON path expression error	Use REGEXP_SUBSTR instead of dynamic PATH in JSON_TABLE.
Chart Empty	Values returned as strings	Use parseFloat and replace commas in JavaScript.
Rate Limited (429)	Free tier limits hit	Switch to inception/mercury-2 paid tier.

The Future of Enterprise Apps:
Natural language meets Oracle APEX. Deep database execution requires no SQL knowledge from the end-user.